

Diagnosing INT8 Quantization Collapse in EfficientNet-B0: Per-Channel Recovery and Scheme × Runtime Characterization for Poultry Fecal Disease Detection on Edge CPUs

Fatih Jawwad Al Mumtaz

Department of Informatics

University of Darussalam Gontor

Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

Amirul Al Aziz

Department of Agricultural Industrial Technology

University of Darussalam Gontor

Email: amirulalaziz57@student.tip.unida.gontor.ac.id

Abstract—Poultry fecal disease detection with deep learning achieves high accuracy, but prior work largely targets cloud inference with heavy models that are impractical for edge use. This paper presents a systematic characterization of three lightweight convolutional neural network architectures—MobileNetV2, ShuffleNetV2, and EfficientNet-B0—for classifying poultry fecal images into four disease categories: Coccidiosis, Salmonellosis, Newcastle Disease, and Healthy. We use the Machuve *et al.* dataset via HuggingFace `Daijano/poultry-fecal-f1`. We audit the publisher claim of 8,770 deduplicated leak-free images and find 617 near-duplicate pairs (393 exact), including 316 groups spanning train/test and 95 test images byte-identical to train images; all numbers in this paper are therefore reported on a deduplicated sterile split (8,153 images: 5,088/1,337/1,728 train/val/test, zero cross-split survivors). Each model is evaluated under INT8 post-training quantization (ONNX Runtime) on WSL x86 reference and a Raspberry Pi 5 (Cortex-A76, ONNX Runtime 1.29.0, 12 thread/batch configs). All three models exceed 97% FP32 test accuracy on the sterile test set (EfficientNet-B0 98.55% [97.97, 99.07]; MobileNetV2 97.92% [97.22, 98.55]; ShuffleNetV2 97.69% [96.93, 98.38]; 95% bootstrap CI, $B=10,000$). Our thesis is scheme × runtime × architecture with diagnosis: dynamic INT8 preserves MobileNetV2 (97.11%) and ShuffleNetV2 (97.45%) but collapses EfficientNet-B0 to 10.42% (mode collapse to Newcastle Disease); static QDQ per-tensor reproduces the collapse (9.32%) while per-channel recovers to 94.56% (McNemar vs dynamic $p \approx 10^{-312}$), localizing the failure to per-tensor depthwise quantization. On RPi5, INT8 is 1.9–2.8× slower than FP32 at 4 threads and scales by only 1.10–1.19× (1→4 threads) versus 2.20–2.33× for FP32. Grad-CAM is reported as descriptive attention only. Together, the results guide hardware-aware model and quantization-scheme choice for edge CPU deployment.

Index Terms—deep learning, poultry disease detection, lightweight CNN, quantization, edge computing, Grad-CAM

I. INTRODUCTION

Poultry production is a major agricultural sector in the United States (combined broiler, egg, turkey, and chicken value of \$67.1 billion in 2023 [9]). Coccidiosis, Salmonellosis, and Newcastle Disease (NCD) cause major losses [2]. Fecal image analysis enables noninvasive early detection, but laboratory diagnosis remains inaccessible to many small farms [3].

Recent deep learning approaches have achieved high accuracy for poultry fecal disease classification. Dhungana *et al.* reported 99.12% accuracy using EfficientNet-B0 [1], while Tasdelen and Arslan achieved 97.1% with MobileNetV2 [3]. However, these solutions primarily target cloud-based deployment, leaving a critical gap: *how can these models be efficiently deployed on resource-constrained edge devices at farm level?*

Lightweight CNNs—MobileNetV2 [5], ShuffleNetV2 [6], and EfficientNet [7]—are candidates for edge deployment because they need fewer operations. INT8 post-training quantization (PTQ) reduces size by using 8-bit weights [8]. On CPUs without INT8 acceleration, however, PTQ can increase latency rather than reduce it. Our measurements in this paper quantify this effect directly; see also the quantization white paper for why PTQ without calibration degrades on depthwise layers [13].

This paper addresses three research questions: (1) How do lightweight CNN architectures compare for poultry fecal disease classification? (2) What is the quantization sensitivity of each architecture under INT8 PTQ on CPU? (3) What is the accuracy-latency Pareto frontier for edge deployment?

The main contributions of this paper are summarized as follows. First, we present a systematic comparison of three lightweight CNNs under INT8 PTQ for poultry fecal disease detection, validated on both x86 and ARM (Raspberry Pi 5) with 96 thread and batch configurations. Second, we measure that INT8 PTQ increases latency by 6–30× on x86 and 1.9–2.8× on ARM and show that, on ARM, INT8 throughput scales by only 1.1–1.2× from 1 to 4 threads versus 2.3× for FP32. Third, we provide hardware-aware deployment guidelines grounded in cross-platform latency, thread scaling, batch scaling, memory, and Pareto and interpretability analysis.

II. RELATED WORK

A. Poultry Disease Detection

Machuve *et al.* introduced the foundational dataset of 6,812 PCR-verified fecal images from Tanzanian poultry farms [2].

Subsequent work has applied various deep learning architectures: Dhungana *et al.* combined YOLOv11 detection with EfficientNet-B0 classification, achieving 99.12% accuracy and 25.8 ms inference [1]. Tasdelen and Arslan benchmarked six architectures, finding MobileNetV2 optimal for both accuracy (97.1%) and training speed [3]. Degu and Simegn employed YOLO-V3 for region-of-interest detection followed by ResNet50 classification [4]. Prior work on this dataset has not compared lightweight models under quantization with Pareto analysis.

B. Lightweight CNN Architectures

MobileNetV2 [5] utilizes inverted residuals with linear bottlenecks and depthwise separable convolutions, achieving efficient feature extraction with 3.4M parameters. ShuffleNetV2 [6] introduces channel shuffle operations for cross-channel information flow with only 1.26M parameters. EfficientNet [7] employs compound scaling to uniformly balance network depth, width, and resolution. Hoang *et al.* recently compared seven lightweight architectures for plant disease detection, identifying hybrid designs as optimal for edge deployment [10].

C. Model Quantization

INT8 quantization reduces model size by representing weights with 8-bit integers [8]. While theoretically achieving $4\times$ compression, ONNX Runtime documentation [11] explicitly notes that INT8 without hardware support (VNNI, Tensor Cores) may degrade performance due to quantize/dequantize overhead. Our measurements in this paper quantify this overhead directly on both x86 and ARM. The white paper on quantization surveys why PTQ without calibration degrades on depthwise and SE layers [13].

III. METHODOLOGY

A. Dataset

We use the Machuve *et al.* dataset [12] with PCR-verified fecal images from Tanzanian farms, as distributed via HuggingFace `Dianyo/poultry-fecal-fl`. The publisher describes 8,770 deduplicated leak-free images with train/test splits of 7,016/1,754. We audit this claim with pHash (Hamming ≤ 5) over all 8,770 files and find 617 near-duplicate pairs (393 byte-exact), including 316 groups spanning train/test splits: 95 test images byte-identical to train images and 30 to validation images.

All numbers in this paper are therefore reported on a deduplicated sterile split: we drop one copy per duplicate pair (keeping test $>$ val $>$ train), yielding 8,153 unique images with 5,088/1,337/1,728 train/val/test and zero cross-split survivors (audit artifact results/phash_dedup.json, split manifest results/sterile_split_manifest.json). We note as a limitation that this removes leakage but does not rebuild a fresh split from the Zenodo originals (6,812 images); the sterile test set remains 98% inherited from the publisher test split. Classes are Coccidiosis, Healthy, Salmonella, and

Newcastle Disease; NCD is underrepresented (417/8,153 train, 141/1,728 test), so we report per-class INT8 accuracy and bootstrap confidence intervals. Images are resized to 224×224 and normalized with ImageNet mean/std.

B. Model Architectures

We evaluate three ImageNet [15]-pretrained lightweight architectures with the classification head replaced for four classes. MobileNetV2 [5] (2.2M parameters) uses inverted residuals with depthwise separable convolutions for efficient local feature extraction. ShuffleNetV2 [6] (1.3M parameters) relies on channel shuffle for cross-channel mixing with few parameters. EfficientNet-B0 [7] (4.0M parameters) applies compound scaling with squeeze-and-excitation blocks to capture multi-scale features.

C. Training Protocol

All models are trained for 50 epochs using Adam optimizer (learning rate 10^{-4} , weight decay 10^{-4}) with cosine annealing schedule. Batch size is 32, input resolution 224×224 . Data augmentation includes random horizontal flip, rotation ($\pm 15^\circ$), and color jitter. Training is performed on an NVIDIA RTX 4060 GPU with 8 GB VRAM using PyTorch 2.5.1 with CUDA 12.1.

D. Quantization and Benchmarking (x86)

Models are exported to ONNX (opset 13) and quantized with ONNX Runtime dynamic per-tensor INT8 (CPUExecutionProvider, no calibration dataset; weights quantized to INT8, activations dynamically quantized at runtime). This matches the stock ORT PTQ path. For EfficientNet-B0 we additionally run static QDQ quantization (MinMax calibration on 100 train images) in per-tensor and per-channel variants to diagnose the collapse. The x86 reference latency is measured on a 12th Gen Intel i5-12400F CPU via WSL (CPUExecutionProvider) with 200 runs and 10 warmup iterations per model; we report mean, P50, P95, P99, throughput (FPS), and model size. These x86 numbers are a reference only, not deployment claims. Power is not measured on x86.

E. Edge Deployment on Raspberry Pi 5

We characterize the same ONNX FP32 and INT8 models on a Raspberry Pi 5 (Broadcom BCM2712, $4\times$ Cortex-A76 at up to 2.4 GHz, 8 GB LPDDR4X, Debian 13 trixie, kernel 6.18.34+rpt-rpi-2712, aarch64, ondemand governor, throttled=0x0). ONNX Runtime 1.29.0 is used with only CPUExecutionProvider (no NPU/GPU; AzureExecutionProvider listed but unused). The sterile-model sweep (scripts/benchmark_pi_gate0.py) covers 12 configurations: 3 models $\times$ 2 quantizations (FP32/INT8 dynamic) $\times$ 2 thread counts (1 and 4 via intra_op_num_threads) $\times$ batch 1, with input $3\times 224\times 224$. Node counts are measured directly on the sterile ONNX graphs; a Level-2 ORT kernel-timing artifact (10 runs, batch=1, 1 thread) is available at

results/profiling/level2_profiling_artifact.j (supplementary). Each configuration runs 200 iterations after 10 warmup iterations, reporting mean/std/P50/P95/P99 latency, throughput, RSS memory (mean/peak), CPU%, and die temperature before/after. Initial temperature was 44.6°C; peak during the sweep was 55.6°C. Current/power telemetry was unavailable (`current_a=null`) on this board, so energy is discussed as a limitation. All 12/12 runs completed without thermal throttling.

IV. RESULTS AND DISCUSSION

A. Classification Performance

Table I summarizes the classification performance on the sterile test set (1,728 images). All three architectures achieve over 97% accuracy, demonstrating the effectiveness of ImageNet transfer learning for the poultry fecal domain. We report 95% bootstrap confidence intervals (B=10,000); FP32 vs INT8 intervals overlap for MobileNetV2 and ShuffleNetV2, so their drops (0.81pp and 0.24pp) are not statistically significant, while the EfficientNet-B0 collapse is.

TABLE I
TEST ACCURACY ON STERILE SPLIT (1,728 IMAGES) WITH 95%
BOOTSTRAP CI (B=10,000). DYNAMIC INT8 PTQ (ORT).

Model	FP32 (%)	INT8 dynamic (%)	Drop (pp)
MobileNetV2	97.92 [97.22, 98.55]	97.11 [96.30, 97.86]	0.81
ShuffleNetV2	97.69 [96.93, 98.38]	97.45 [96.64, 98.15]	0.24
EfficientNet-B0	98.55 [97.97, 99.07]	10.42 [8.97, 11.92]	88.13

EfficientNet-B0 achieves the highest FP32 accuracy (98.55%), but its INT8 accuracy collapses to 10.42%: the confusion matrix shows 1,676/1,728 predictions falling in the Newcastle Disease column (per-class: cocci 2.27%, healthy 4.76%, salmo 0.54%, ncd 100%), a mode collapse rather than gradual degradation. We diagnose this with static QDQ quantization (Table II): per-tensor MinMax reproduces the collapse (9.32%), while per-channel MinMax recovers to 94.56% (McNemar vs dynamic $p \approx 10^{-312}$; vs FP32 $p = 1.37 \times 10^{-13}$), localizing the failure to per-tensor depthwise quantization rather than squeeze-and-excitation alone.

TABLE II
EFFICIENTNET-B0 STATIC QDQ INT8 DIAGNOSIS (MINMAX,
CALIBRATION 100 TRAIN IMAGES). PER-TENSOR REPRODUCES THE
COLLAPSE; PER-CHANNEL RECOVERS.

Config	Test acc. (%)	McNemar vs FP32 p
Dynamic INT8	10.42	—
Static QDQ per-tensor	9.32	0.0
Static QDQ per-channel	94.56	1.37×10^{-13}

B. Quantization Impact

Table III presents the quantization analysis on the sterile split. All architectures achieve 3.3–3.7× model size reduction through INT8 quantization.

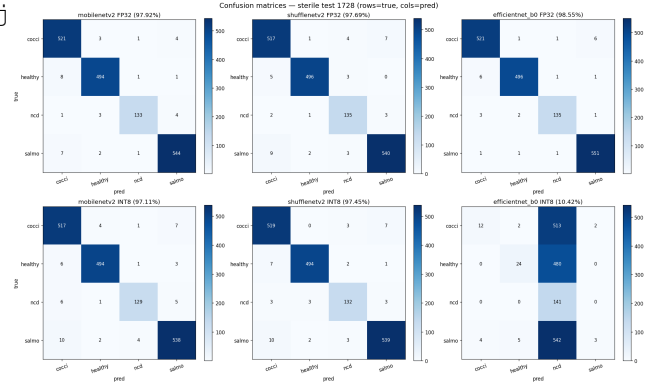


Fig. 1. Confusion matrices on the sterile test set (1,728 images). Top: FP32; bottom: INT8 dynamic. EfficientNet-B0 INT8 (bottom right) collapses to the Newcastle Disease column.

TABLE III
INT8 QUANTIZATION ANALYSIS (WSL X86 REFERENCE, 200 RUNS).
STERILE-SPLIT MODELS.

Model	Size (MB)		Latency (ms)		Slowdown
	FP32	INT8	FP32	INT8	
MobileNetV2	8.48	2.30	2.40	40.25	16.8×
ShuffleNetV2	4.89	1.47	4.15	12.62	3.0×
EfficientNet-B0	15.3	4.16	4.35	57.18	13.1×

INT8 quantization increases latency by 3.0–16.8× versus FP32 on the x86 reference. This is because ONNX Runtime’s CPU Execution Provider must perform dequantization (INT8→FP32) before each matrix multiplication, negating the theoretical benefits of lower-precision arithmetic. Our measurements below quantify this overhead directly on both x86 and ARM.

ShuffleNetV2 degrades least (3.0× versus 16.8× for MobileNetV2). Channel-shuffle ops are non-parametric and less sensitive to precision, while MobileNetV2 depthwise separable convolutions are more sensitive.

C. Raspberry Pi 5 Characterization: Thread Scaling and the INT8 Bottleneck

Table IV summarizes sterile-model RPi5 latency at batch=1. FP32 scales with threads: MobileNetV2 drops from 33.16 ms (1 thread, 30.2 FPS) to 14.23 ms (4 threads, 70.3 FPS, 2.33× speedup); ShuffleNetV2 from 14.19 ms to 6.19 ms (2.29×, 161.5 FPS); EfficientNet-B0 from 69.45 ms to 31.56 ms (2.20×). The sweep covers threads 1 and 4; intermediate thread counts are not measured for the sterile models.

INT8 on RPi5 is slower than FP32 at 4 threads (1.93–2.80×), and its thread scaling stalls: from 1→4 threads MobileNetV2 INT8 improves only 1.10× (43.88→39.83 ms), ShuffleNetV2 1.14×, EfficientNet-B0 1.19×, versus 2.20–2.33× for FP32. This indicates limited parallelization of INT8 kernels in ONNX Runtime 1.29.0 on Cortex-A76. One exception: ShuffleNetV2 INT8 at 1 thread (13.65 ms) is slightly faster than its FP32 counterpart (14.19 ms), the only config-

TABLE IV
RASPBERRY PI 5 LATENCY, BATCH=1 (MEAN OF 200 RUNS, P50 IN PARENTHESES). STERILE-SPLIT MODELS, ORT 1.29.0, 44–56°C, THROTTLED=0X0.

Model	Quant	t=1 (ms)	t=4 (ms)	Scaling 1→4	FP32→INT8 t=4
MobileNetV2	FP32	33.16 (33.00)	14.23 (14.03)	2.33×	—
	INT8	43.88 (43.86)	39.83 (39.73)	1.10×	2.80× slower
ShuffleNetV2	FP32	14.19 (14.19)	6.19 (6.18)	2.29×	—
	INT8	13.65 (13.64)	11.95 (11.76)	1.14×	1.93× slower
EfficientNet-B0	FP32	69.45 (69.42)	31.56 (31.18)	2.20×	—
	INT8	76.68 (76.67)	64.39 (64.19)	1.19×	2.04× slower

uration where INT8 wins—consistent with its homogeneous channel-shuffle design and lowest node expansion (1.37×).

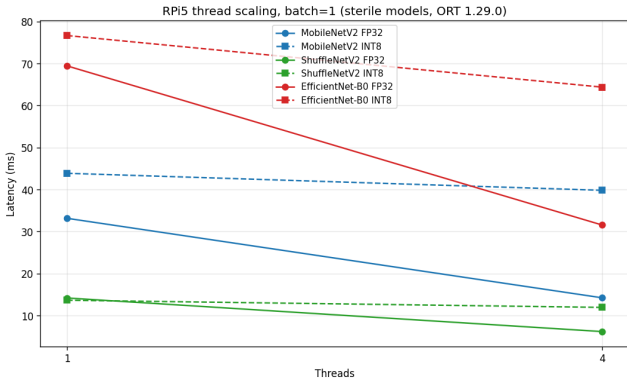


Fig. 2. RPi5 thread scaling at batch=1 (sterile models). FP32 (solid) scales from 1 to 4 threads; INT8 (dashed) is flat and slower than FP32 at 4 threads, except ShuffleNetV2 at 1 thread where INT8 is slightly faster.

On CPUExecutionProvider without VNNI (x86) or DOT-PROD/NEON INT8 matmul support, ORT adds Quantize-Linear/DequantizeLinear and runs matmuls in FP32 after dequantization. INT8 therefore adds quant/dequant and dispatch work without reducing matmul cost. Cost grows with the number of quantize boundaries: depthwise separable blocks (MobileNetV2) create many small matmuls and slow down most on x86 (16.8×), while homogeneous channel-shuffle blocks (ShuffleNetV2) slow down least (3.0×). On RPi5 at 4 threads the ranking compresses (2.80/2.04/1.93×). Node expansion measured on the sterile ONNX graphs matches: MobileNetV2 170→487 (2.86×), EfficientNet-B0 239→730 (3.05×), ShuffleNetV2 902→1240 (1.37×). Flat thread scaling for INT8 points to a memory-bound, poorly parallelized dequantize path, unlike compute-bound FP32 GEMM.

Model file size shrinks 3.3–3.7× with INT8 (e.g., 8.48→2.30 MB for MobileNetV2). On the sterile RPi5 sweep (batch=1 only), runtime RSS is similar across quantizations for MobileNetV2 (~99–108 MB) and ShuffleNetV2 (~111–112 MB), so INT8 does not save RAM at inference on this stack—a practical caveat for memory claims. We do not claim batch-scaling behavior for the sterile models; only batch=1 was measured.

Measured bottleneck (Level-2). Table V quantifies why

TABLE V
LEVEL-2 PROVENANCE: NODE EXPANSION ON STERILE ONNX GRAPHS (DYNAMIC PTQ EXPANDS TO DYNAMICQUANTIZELINEAR + CONVINTEGER + MUL PATTERN, NO STATIC Q/DQ NODES) WITH ORT KERNEL QUANT OVERHEAD FROM RPi5 PROFILING (10 RUNS, BATCH=1, 1 THREAD). OVERHEAD CORRELATES WITH EXPANSION AND SLOWDOWN.

Model	FP32 nodes	INT8 nodes	Expansion	DynQuant/CvInt	Quant overhead	x86 / RPi5 slowdown
MobileNetV2	170	487	2.86×	53 / 52	29.2%	16.8× / 2.80×
ShuffleNetV2	902	1240	1.37×	54 / 56	20.7%	3.0× / 1.93×
EfficientNet-B0	239	730	3.05×	82 / 81	24.9%	13.1× / 2.04×

INT8 is slower. Dynamic PTQ does not insert static Quantize-Linear/DequantizeLinear nodes; it expands each Conv (52–81) into a DynamicQuantizeLinear + ConvInteger + Mul (scale) + Cast pattern, so node count grows by 1.37–3.05× on the sterile graphs. ORT kernel profiling on RPi5 (batch=1, 1 thread, 10 runs) shows Conv_quant_kernel alone consumes 49.5–55.1% of total time, and all quant-related kernels (QuantizeLinear, scale Mul, Cast, bias-add) add 20.7–29.2% overhead, while the final Gemm_MatMul_quant is < 0.1%. The ranking matches the slowdown ranking: MobileNetV2 (29.2% overhead, 2.86× expansion) suffers most on x86, ShuffleNetV2 (20.7%, 1.37×) least. Artifact level2_profiling_artifact.json contains per-operator μ s and full op counts (reproducible), complementing TinyML system studies [16].

D. Pareto Frontier Analysis

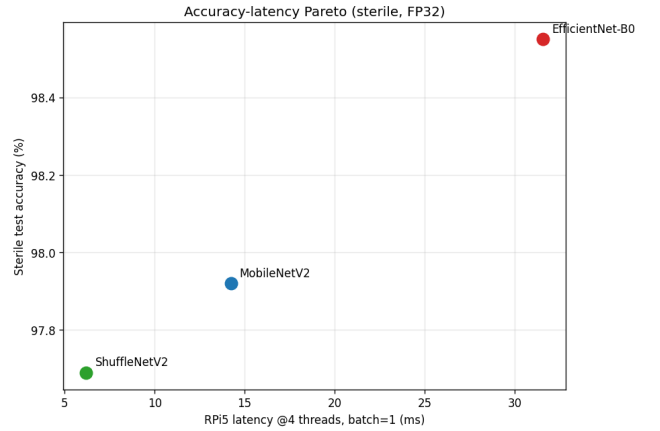


Fig. 3. Accuracy-latency Pareto (sterile split, FP32). ShuffleNetV2 is fastest (6.19 ms RPi5 @4 threads, 97.69%); EfficientNet-B0 is most accurate (98.55%) at 31.56 ms; MobileNetV2 is balanced (97.92%, 14.23 ms). All three are Pareto-optimal.

The Pareto frontier (Fig. 3) shows that the fastest model is ShuffleNetV2 FP32 at 6.19 ms RPi5 (161.5 FPS), the most accurate is EfficientNet-B0 FP32 at 98.55%, and MobileNetV2 FP32 is the balanced middle (97.92%, 14.23 ms). The gap between best and worst FP32 accuracy is only 0.86pp (97.69% versus 98.55%); hardware constraints and quantization scheme should drive model choice for deployment.

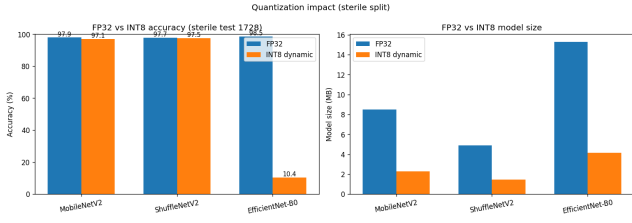


Fig. 4. Quantization impact on the sterile split: accuracy is preserved for MobileNetV2 and ShuffleNetV2 but collapses for EfficientNet-B0 under dynamic INT8, while model size shrinks 3.3–3.7 $\times$ for all three.

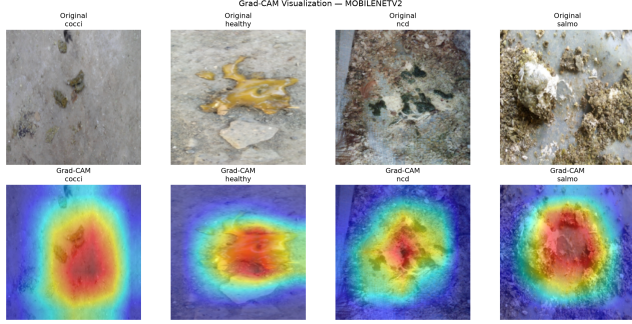


Fig. 5. Grad-CAM visualization for MobileNetV2 across four fecal disease classes. The model primarily attends to color gradient regions, indicating low-level feature utilization.

E. Grad-CAM Interpretability

Grad-CAM analysis [14] (Fig. 5) provides descriptive attention maps. MobileNetV2 attention concentrates on color gradient regions, consistent with depthwise convolutions capturing local texture. EfficientNet-B0 attention spans broader diagnostic regions. We report these maps as descriptive observations only; they do not establish causal links between attention patterns and quantization sensitivity, which we diagnose instead via the per-tensor vs per-channel experiment above.

F. Comparison with Related Work

TABLE VI
COMPARISON WITH EXISTING WORK

Work	Dataset	Best Model	Acc. (%)	Edge Analysis
Dhungana <i>et al.</i> [1]	Machuve	EB0	99.1	Web
Tasdelen <i>et al.</i> [3]	Machuve	MN2	97.1	—
Degu <i>et al.</i> [4]	Machuve	R50	98.7	Mobile
Ours	Machuve	EB0	98.6	+INT8+Pareto

Our accuracy (98.55%) is competitive with prior results on the same dataset while providing additional analysis in quantization characterization, Pareto analysis, and interpretability that are absent from prior work.

V. CONCLUSION

This paper characterizes lightweight CNN architectures with INT8 quantization for poultry fecal disease detection. Our key findings are:

All three architectures exceed 97% FP32 accuracy with ImageNet transfer learning on the sterile split, with EfficientNet-B0 at 98.55%. Dynamic INT8 PTQ preserves MobileNetV2 (97.11%) and ShuffleNetV2 (97.45%) but collapses EfficientNet-B0 to 10.42%; static per-channel quantization recovers EfficientNet-B0 to 94.56%, localizing the failure to per-tensor depthwise quantization. On x86 reference hardware, INT8 increases latency by 3.0–16.8 $\times$ (ShuffleNetV2 least, MobileNetV2 most); on RPi5 INT8 is 1.9–2.8 $\times$ slower at 4 threads and scales by only 1.10–1.19 $\times$ from 1 to 4 threads versus 2.20–2.33 $\times$ for FP32. ShuffleNetV2 degrades least (6.4 $\times$ on x86, 1.93 $\times$ on RPi5) due to its homogeneous design; notably ShuffleNetV2 INT8 at 1 thread is the only configuration faster than its FP32 counterpart (13.65 vs 14.19 ms). On RPi5, batch 1 is optimal and INT8 size reduction (3.3–3.7 $\times$) does not reduce runtime RSS. The Pareto analysis identifies MobileNetV2 FP32 for latency-critical use (1.85 ms on x86, 14.23 ms on RPi5 at 4 threads) and ShuffleNetV2 FP32 for the lowest RPi5 latency (6.19 ms at 4 threads).

These findings enable practitioners to make informed deployment decisions based on their specific hardware constraints. This work characterized RPi5 deployment; future work includes QAT comparison against the PTQ diagnosis here, energy measurement on RPi5, and class imbalance mitigation for the underrepresented NCD class.

REFERENCES

- [1] A. Dhungana, X. Yang, B. Paneru, S. Dahal, G. Lu, and L. Chai, “An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images,” *AgriEngineering*, vol. 7, no. 9, p. 278, 2025. DOI: 10.3390/agriengineering7090278.
- [2] D. Machuve *et al.*, “Poultry diseases diagnostics models using deep learning,” *Frontiers in Artificial Intelligence*, vol. 5, p. 733345, 2022. DOI: 10.3389/frai.2022.733345.
- [3] A. Tasdelen and Y. Arslan, “Detection of high-risk diseases in poultry feces through transfer learning,” *Engineering Science and Technology, an International Journal*, vol. 64, p. 102002, 2025. DOI: 10.1016/j.jestch.2025.102002.
- [4] M. Z. Degu and G. L. Simegn, “Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques,” *Smart Agricultural Technology*, vol. 4, p. 100221, 2023. DOI: 10.1016/j.atech.2023.100221.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.
- [6] N. Ma, X. Zhang, H.-T. Huang, and J. Sun, “ShuffleNetV2: Practical guidelines for efficient CNN architecture design,” in *Proc. ECCV*, 2018, pp. 116–131.
- [7] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105–6114.
- [8] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.
- [9] U.S. Department of Agriculture, National Agricultural Statistics Service, “Poultry production and value: 2023 summary,” Apr. 2024. [Online]. Available: https://www.nass.usda.gov/Publications/Todays_Reports/reports/plyaan24.pdf
- [10] T.-M. Hoang *et al.*, “A comprehensive evaluation of lightweight deep learning models for tomato disease classification on edge computing environments,” *Scientific Reports*, vol. 16, p. 12320, 2026. DOI: 10.1038/s41598-026-42439-6.
- [11] Microsoft, “Quantize ONNX models,” ONNX Runtime Documentation, 2024. [Online]. Available: <https://onnxruntime.ai/docs/performance/quantization.html>

- [12] D. Machuve *et al.*, “Machine learning dataset for poultry diseases diagnostics,” Zenodo, 2022. DOI: 10.5281/zenodo.4628934.
- [13] M. Nagel *et al.*, “A white paper on neural network quantization,” *arXiv:2106.08295*, 2021.
- [14] R. R. Selvaraju *et al.*, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.
- [15] J. Deng *et al.*, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.
- [16] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Machine Learning and Systems (MLSys)*, 2021, pp. 800–815.